

Core Python Syllabus

Introduction to Python

1. Overview of Python

- What is Python?
- History and evolution of Python
- Python versions (Python 2 vs Python 3)
- Importance of Python in various fields

2. Setting Up Python Environment

- Installing Python (Windows, macOS, Linux)
- Python IDEs and text editors (IDLE, PyCharm, VS Code, Jupyter Notebook)
- Writing and running Python scripts
- Python Interactive Shell

Python Basics

1. Syntax and Semantics

- Basic syntax and structure of Python programs
- Comments and documentation
- Indentation and blocks

2. Variables and Data Types

- Declaring variables
- Built-in data types:
 - Numbers: int, float, complex
 - Strings: str
 - Booleans: bool
 - NoneType: None
- Type conversion and casting

3. Operators

- Arithmetic operators: +, -, *, /, //, %, **
- Comparison operators: ==, !=, >, <, >=, <=
- Logical operators: and, or, not
- Bitwise operators: &, |, ^, ~, <<, >>
- Assignment operators: =, +=, -=, *=, /=, //=, %=, **=

4. Input and Output

- Taking user input: input()
- Printing output: print()
- String formatting:
 - Using f-strings: f"Hello, {name}"
 - format() method: "Hello, {}".format(name)
 - % formatting: "%s" % (name,)

Control Structures

1. Conditional Statements

- if, elif, else statements
- Nested conditions
- Short-hand if-else

2. Loops

- for loops: `for item in iterable:`
- while loops: `while condition:`
- Loop control statements: `break`, `continue`, `pass`
- Iterating with `enumerate()`, `zip()`, `range()`

3. Comprehensions

- List comprehensions: `[expr for item in iterable if condition]`
- Dictionary comprehensions: `{key_expr: value_expr for item in iterable if condition}`
- Set comprehensions: `{expr for item in iterable if condition}`
- Generator expressions: `(expr for item in iterable if condition)`

Functions and Modules

1. Defining Functions

- Function syntax and definition
- Parameters and arguments
- Return values
- Scope and lifetime of variables
- Lambda functions: `lambda parameters: expression`

2. Advanced Function Concepts

- Default arguments
- Keyword arguments
- Variable-length arguments: `*args`, `**kwargs`
- Anonymous functions: `lambda`

3. Modules and Packages

- Importing modules: `import`, `from ... import`
- Standard library modules:
 - `math`, `random`, `datetime`, `os`, `sys`, `json`, `csv`
- Creating custom modules
- Organizing code with packages

Data Structures

1. Lists

- Creating and accessing lists
- List methods and operations:
 - `append()`, `extend()`, `insert()`, `remove()`, `pop()`, `index()`, `count()`, `sort()`, `reverse()`
- Slicing and indexing
- List comprehensions

2. Tuples

- Creating and accessing tuples
- Tuple methods and operations:
 - `count()`, `index()`

- Packing and unpacking tuples

3. Sets

- Creating and accessing sets
- Set methods and operations:
 - `add()`, `remove()`, `discard()`, `pop()`, `clear()`, `union()`, `intersection()`, `difference()`, `symmetric_difference()`
- Set comprehensions

4. Dictionaries

- Creating and accessing dictionaries
- Dictionary methods and operations:
 - `keys()`, `values()`, `items()`, `get()`, `pop()`, `popitem()`, `clear()`, `update()`
- Dictionary comprehensions

Strings

1. String Operations

- Creating and accessing strings
- String methods and functions:
 - `len()`, `capitalize()`, `lower()`, `upper()`, `title()`, `swapcase()`, `strip()`, `lstrip()`, `rstrip()`, `center()`, `ljust()`, `rjust()`, `find()`, `index()`, `replace()`, `split()`, `join()`, `partition()`, `rpartition()`, `format()`
- String slicing and indexing

2. String Formatting

- Using f-strings: `f"Hello, {name}"`
- `format()` method: `"Hello, {}".format(name)`
- % formatting: `"%s" % (name,)`

3. Regular Expressions

- Introduction to regular expressions
- Using `re` module:
 - `re.match()`, `re.search()`, `re.findall()`, `re.finditer()`
 - `re.compile()`, `re.sub()`, `re.split()`

File Handling

1. Reading and Writing Files

- File modes: `read`, `write`, `append`
- Using `with` statement for file operations
- Reading and writing text files
- Working with binary files

2. File Methods

- Common file methods:
 - `read()`, `readline()`, `readlines()`, `write()`, `writelines()`
- File pointers and `seek()` method

3. CSV and JSON

- Working with CSV files using `csv` module

- Working with JSON data using json module

Error Handling and Exceptions

1. Introduction to Exceptions

- What are exceptions?
- Common built-in exceptions

2. Handling Exceptions

- Using try, except blocks
- Multiple except blocks
- else and finally blocks
- Raising exceptions: raise
- Creating custom exceptions

Object-Oriented Programming (OOP)

1. Introduction to OOP

- Basic principles of OOP: encapsulation, inheritance, polymorphism
- Classes and objects

2. Defining Classes

- Class syntax and definition
- Attributes and methods
- `__init__` method

3. Inheritance

- Creating subclasses
- Method overriding
- Using `super()`

4. Polymorphism

- Method overloading
- Method overriding
- Duck typing

5. Special Methods

- Operator overloading
- Special methods:
 - `__str__()`, `__repr__()`, `__len__()`, `__call__()`, `__getitem__()`, `__setitem__()`, `__delitem__()`, `__iter__()`, `__next__()`

6. Encapsulation

- Private and protected members
- Getter and setter methods
- property decorator

Advanced Python Concepts

1. Decorators

- Function decorators
- Class decorators
- Using @property, @staticmethod, @classmethod

2. Generators and Iterators

- Creating iterators using __iter__, __next__
- Using generators with yield
- Generator expressions

3. Context Managers

- Using with statement
- Creating custom context managers with __enter__, __exit__

4. Multithreading and Multiprocessing

- Introduction to threading
- Creating and managing threads
- Introduction to multiprocessing
- Creating and managing processes

Standard Library and Third-Party Modules

1. Working with Dates and Times

- Using datetime module
- Formatting and parsing dates
- Working with time zones

2. Networking

- Using socket module
- Creating client-server applications

3. Web Scraping

- Introduction to web scraping
- Using requests module
- Parsing HTML with BeautifulSoup

Debugging

1. Debugging

- Debugging tools and techniques
- Using pdb for interactive debugging

Project Work

1. Building a Command-Line Application

- Planning and designing the application
- Implementing core functionality
- Adding error handling and logging

2. Developing a Web Scraper

- Identifying target websites and data
- Fetching and parsing web pages
- Storing and processing scraped data